Sol Current-State Audit

Remaining Work, Risks, and the Shortest End-to-End Path

Roadmap state after E1c - August 25, 2026

1 Executive Assessment

Sol currently has a credible end-to-end **executable language model**:

```
source/package
-> lexer and parser
-> syntax validation
-> HIR/name resolution
-> type/effect/contract analysis
-> owning typed IR
-> ownership validation
-> reference interpreter
-> sol test
```

This is real functionality rather than scaffolding. Records, enums, tuples, bounded generics, traits, effects, capabilities, contracts-as-obligations, ownership, mutation, loops, handlers, recursive patterns, cleanup, and host operations cross the complete pipeline.

Sol now has a bounded end-to-end interpreter application profile with explicit entrypoints, trusted host capabilities, executable core contracts/refinements, and `sol run`. It is not yet a production application toolchain: there is no MIR, native or WebAssembly backend, linker, target runtime ABI, `sol build`, package manifest, dependency system, or public semantic IR.

Current verification is healthy:

- Normal test suite: 37/37 passed.
- Clean ASan/UBSan suite: 37/37 passed.
- Worktree was clean except the intentionally untracked session file.

2 Current State

2.1 Source, Parsing, and Packages

The compiler has deterministic single-file and directory-package loading, a lossless lexer, a recovering arena parser, exhaustive syntax-tree validation, canonical formatting, multi-file modules, imports, and stable top-level semantic IDs.

The package model remains intentionally bounded. It has no manifests, dependency graph, lockfile, feature policy, aliases, re-exports, package-qualified identity, or host configuration. Directory packages are aggregated into package-global syntax and semantic arenas.

2.2 Semantic Pipeline

Name resolution, bounded first-order generics, records, enums, tuples, structural function types, traits, method evidence, effects, capabilities, exact handlers, contracts, recursive patterns, and exhaustiveness are operational.

The current HIR is primarily syntax-indexed semantic metadata rather than an independent typed semantic representation. Types, effects, contracts, and resolutions are maintained in parallel tables indexed by syntax IDs. This is effective for the bootstrap but will complicate incremental compilation and public semantic APIs.

2.3 Ownership and IR

The compiler lowers successful programs to a genuine self-contained owning typed IR. The IR retains executable types, calls, places, effects, contract templates, cleanup, pattern trees, and dispatch metadata. Ownership

checking covers affine moves, structural Copy, loans, partial moves, definite initialization, projected mutation, `inout` writeback, loops, regions, and deterministic cleanup.

This IR is an unstable interpreter IR. It is not a control-flow MIR, stable serialized Sol IR, target-independent layout IR, or safe hostile-input format.

2.4 Interpreter and CLI

The deterministic reference interpreter executes the bounded core with checked arithmetic, aggregate values, functions, callbacks, traits, capabilities, exact handlers, mutation, loops, patterns, cleanup, host operations, and explicit resource limits.

The implemented commands are `sol check`, `sol test`, `sol run`, `sol effects`, `sol inspect`, and `sol fmt`. `sol run` executes one explicit package entrypoint after frontend teardown with bounded console, argument, and deterministic configuration capabilities. There is no `sol build` or executable artifact.

2.5 Contracts and Verification

Contracts, refined predicates, loop invariants, decreases measures, snapshots, and obligations are parsed, typed, purity-checked, and retained as deterministic templates. They are not enforced or discharged. The interpreter's contract-check policy exists at the API level but is unsupported.

This is the largest semantic gap relative to Sol's central claim: accepted contract syntax currently means that an obligation is well-formed, not that it has been proved or checked at runtime.

3 Principal Risks and Pitfalls

3.1 Runtime Contracts Are Absent

Preconditions, postconditions, refinements, loop invariants, decreases clauses, and unreachable proofs are not enforced during execution. A program may compile and pass ordinary tests while violating a declared contract. Documentation currently discloses this, but tooling and users can still overinterpret contract acceptance.

Priority: high. Runtime contract semantics should be implemented before Sol is presented as an application language with progressive verification.

3.2 Application Execution Boundary Is Incomplete

The compiler defines argumentless `@entry`, package uniqueness, its bounded signature, capability-only parameters, successful result-to-status mapping, owning-IR/opaque-handle metadata, and trusted capability injection through an exact root/member registry. `sol run` adds missing-entry and status-boundary diagnostics plus stable panic/runtime rendering.

This keeps later interpreter and backend work from embedding accidental function-name or truncating exit-status conventions.

3.3 Bounded Host Failure Text

The raw host callback writes failure text into interpreter-owned fixed-capacity storage with an authoritative length. Empty failures select a stable default; embedded NULs and oversized lengths are rejected. No borrowed failure C string is scanned after callback return.

The raw callback still receives raw IR and remains a trusted compiler-test interface. Ordinary opaque-handle interpretation rejects it; hosted entrypoint execution uses a separate data-only callback and exact opaque registry that expose no IR or authority token.

3.4 Public C IR Is Not a Hostile-Input Format

The public C IR structure exposes mutable pointers and counts. Logical validation cannot prove that a non-null pointer actually addresses the advertised number of objects. Internally generated IR is strongly validated, but arbitrary foreign or corrupted C metadata cannot be made memory-safe by logical validation alone.

A future public serialized IR requires a separate checked decoder and owned representation.

3.5 Bounded Compilation Resources

Compilation sessions enforce configurable per-file and package bytes, source-file count, directory depth and visited entries, tokens, persistent compiler arenas, diagnostics, cumulative allocation bytes, and allocation-request limits. The first exhausted resource produces a deterministic resource failure and no validated handle.

Specialized parser and semantic recursion ceilings remain in addition to the session-wide meter.

3.6 Package Filesystem Races

Directory traversal is anchored to verified open descriptors and uses descriptor-relative inspection/opening. Source reads reject symlinks and non-regular files, match discovered device/inode identity, probe EOF, and recheck identity, size, and nanosecond modification/change timestamps.

Duplicate source identities are rejected. Formatter commit-time transaction hardening remains distinct from compilation loading.

3.7 Sparse Generic Call Analysis

Generic recursion analysis uses sparse outgoing/incoming adjacency and iterative strongly connected components with linear graph storage. Effect inference reuses its call-graph SCCs and packed ascending component members while retaining deterministic monotonic fixed-point rounds.

3.8 Repeated Whole-IR Validation

Lowering retains independent pre-ownership and final validation. The opaque validated handle then serves as the immutable validation certificate for repeated interpretation and effects rendering, eliminating per-test whole-IR rescans.

Public raw mutable-IR interpreter and effects APIs still validate every call, preserving malformed-table test and trust boundaries.

3.9 Syntax-Indexed Side Tables

HIR, types, effects, and contracts are broad parallel tables indexed by syntax IDs. Every new construct requires coordinated updates across many arenas and validators. This architecture also forces package-global invalidation and makes caching or serialization difficult.

The owned compilation session now centralizes phase order and teardown. A more coherent typed semantic layer remains desirable before incremental compilation or public semantic tooling.

3.10 Duplicated Semantic Rules

Independent validation is valuable, but several semantic algorithms exist in parallel frontend and IR forms: match usefulness, finite inhabitation, contract purity, capability roots, callable effects, and ownership-related type recursion. Drift can either accept unsound metadata or reject valid frontend output as malformed IR.

Shared specifications, generated tables, or common constructor-domain utilities should define policy while independent validators continue to verify representation-specific facts.

3.11 Portability and Release Infrastructure

The bootstrap explicitly remains POSIX-only. Checked-in CI performs clean warning-as-error GCC and Clang ASan/UBSan builds. Optional Clang/libFuzzer parser, package, and bounded scalar-IR harnesses have fixed corpora and smoke runs. Scheduled deterministic stress cases track hashes, functional outcomes, and broad Linux timing ceilings.

Install/export targets, release packaging, coverage gates, and a platform abstraction remain future work.

3.12 Testing Gaps

The handwritten positive, negative, sanitizer, malformed-metadata, parser/package/scalar-IR fuzz, and sparse/package/repeated-execution stress coverage is strong. Missing categories include runtime-value fuzzing, systematic allocation-failure injection, generated semantic properties, standard JSON Schema validation, and interpreter/backend differential execution.

4 Remaining Roadmap

The unfinished roadmap falls into five groups:

- Items 28-39: closures, richer traits, constants, numerics, arrays, lifetimes, resources, collections, and iterators.
- Items 40-44: manifests and dependencies, schemas, general effect polymorphism, cross-cutting semantic integration, and unsafe.
- Items 45-50: MIR, FFI, backend, runtime ABI, entrypoints/build/run, and executable contracts.
- Items 51-58: generated properties, SMT, formatter completion, public IR, language server, semantic patches, reports, and agent context.
- Items 59-61: concurrency, reflection/build transforms, and general handlers.

The current numeric order is not the shortest path to usable execution. Most language-breadth work in items 28-44 is not required to execute an application through the existing reference interpreter.

5 Recommended Endpoint

The shortest meaningful endpoint is a bounded `sol run` profile using the existing owning IR and reference interpreter, with explicit entrypoint and host-capability semantics, followed by executable runtime contracts.

This endpoint is not a production runtime. It is the smallest vertical slice that turns Sol from a language-test harness into an executable application model while exercising ownership, effects, authority, contracts, packages, diagnostics, and runtime behavior together.

6 Shortest End-to-End Path

6.1 Milestone 1: Shared Compilation Session and Hardening

Create one reusable compilation/session API instead of keeping orchestration private to the CLI and duplicating it in tests.

Exit criteria:

- Compile one file or package into validated immutable owning IR.
- Request check, inspection, effect, test, or run outputs through one phase owner.
- Centralize initialization, teardown, diagnostics, and phase ordering.
- Add package limits for bytes, files, depth, tokens, arenas, diagnostics, and allocation.
- Replace the unsafe host error pointer with an owned or length-delimited error.
- Replace cubic generic transitive closure with SCC analysis.
- Validate immutable IR once for repeated execution.

6.2 Milestone 2: Entrypoint and Application ABI

Split entrypoint semantics out of roadmap item 49 rather than waiting for a backend.

Define:

- An explicit entrypoint declaration or annotation.
- Visibility and uniqueness rules.
- Allowed generic, parameter, return, effect, and contract shapes.
- Process exit-status mapping.
- Panic and runtime-failure behavior.
- Capability parameter injection.
- Default execution limits.
- Deterministic duplicate or missing-entry diagnostics.

A bounded shape could resemble:

```
@entry
public function launch(console: capability Console) -> Int64
effects { console.write<console> } {
    console.write("hello")
    return 0
}
```

Entrypoint identity should be retained explicitly in owning IR rather than inferred from a function-name scan.

> IMPLEMENTED Argumentless `@entry` is package-unique when present and retained through syntax, HIR, and owning IR. It requires a public nongeneric free function with a body, explicit closed effects, owned nongeneric root-capability parameters, and exact `()` or `Int64` result. The opaque validated-IR handle resolves the marker and maps only successful `()` to 0 or in-range `Int64` to the identical 0-through-255 process status. Panic and other runtime failures remain structured interpreter failures. Ordinary compilation permits entrypoint absence for libraries; `sol run` will require one.

6.3 Milestone 3: Minimal Trusted Host Profile

Split interpreter host support out of roadmap item 48.

Implement only what a representative application requires:

- Console output.
- A bounded process-argument representation.
- Optional deterministic configuration input.
- Explicit capability-root registry.
- Operation allowlist.
- Stable owned host failures.
- Default and configurable execution limits.

Filesystem, network, clock, randomness, and process mutation should remain absent until each receives an explicit authority and deterministic-test policy.

> IMPLEMENTED A registry bound to one validated handle gives every entrypoint capability parameter a distinct opaque root and grants bodyless data-only members per exact root. Preflight rejects missing roots, missing or extra authority, ambiguous effect families, and cross-handle registries before Sol execution. Safe callbacks receive no IR, callable ID, root, or private source. The minimal conventions cover console `write`, immutable arguments `count`/`get`, and deterministic configuration `read`; callback results consume existing value/text budgets, dispatch consumes the host-call budget, and failures are copied into owned diagnostics.

6.4 Milestone 4: `sol run`

Add:

```
sol run <file.sol|package-directory>
```

Required behavior:

- Compile through the shared session.
- Resolve exactly one entrypoint.
- Verify all required host capabilities before execution.
- Free frontend state before running owning IR.
- Execute under deterministic limits.
- Map return, panic, contract failure, and runtime failure to process status.
- Emit human diagnostics and a versioned JSON runtime envelope.

At this point Sol has meaningful end-to-end application execution.

> IMPLEMENTED `sol run` accepts one file or directory package, optional deterministic `--config=KEY=VALUE` entries, and bounded application arguments after `--`. It compiles through the shared session, transfers owning IR before execution, grants only exact Console/Arguments/Configuration signatures and effects, rejects unsupported authority before side effects, and executes core contracts. Human mode preserves exact console bytes on stdout and sends boundary/runtime diagnostics to stderr. JSON mode emits one `sol.run-result` version-1 envelope with base64 console data, symbolic diagnostics, and exact returned status. Successful `()`/`Int64` results map to 0 or identical 0-through-255 status; missing entrypoints, invalid status values, host-profile failures, and runtime failures use driver status 1, while usage uses 2.

6.5 Milestone 5: Executable Contracts

Split runtime contract checking out of roadmap item 50.

Implement:

- `requires` before callable body execution.
- `old` snapshots at callable entry.
- `ensures` after successful return.
- `Result` success/failure outcome clauses.
- Runtime refined-value construction and validation.
- Structured contract-failure diagnostics.
- Exact ownership cleanup when a contract fails.

Loop invariant and decreases checking may follow as a second increment if needed.

> IMPLEMENTED The interpreter CHECK policy executes source-ordered callable preconditions, entry snapshots, always/success/failure postconditions, and bodyless hosted-member contracts under shared deterministic limits. Direct refined construction always validates its type-owned predicate. False predicates have dedicated structured diagnostics, predicate runtime failures retain their original diagnostic, and every exit performs exact ordinary-binding cleanup without exposing logical copies to cleanup observers. `sol run` and `sol test` request CHECK; explicit interpreter callers may retain IGNORE compatibility. Loop invariant and decreases templates remain runtime-erased.

6.6 Milestone 6: Representative Application and Conformance

Add one multi-file executable fixture exercising imports, semantic IDs, records, enums, tuples, generics, traits, recursive matching, ownership, mutation, `Option`/`Result`, capability-injected console output, contracts, runtime failures, and cleanup.

The same fixture should pass through `sol fmt`, `sol check`, `sol test`, `sol effects`, `sol inspect`, and `sol run`. This becomes the acceptance test for the versioned executable-core profile.

7 Recommended Immediate Sequence

```
current item 27
-> shared compilation session and hardening
-> entrypoint/application ABI
-> minimal interpreter host profile
-> sol run
-> executable contracts/refinements
-> representative application and conformance suite
```

Items 28-47 can mostly be deferred for this endpoint.

8 Production Path

After interpreter-based application execution stabilizes the semantics, the shortest production route is likely WebAssembly-first:

```
entrypoint/application ABI
-> ownership-explicit CFG/MIR for the current subset
-> monomorphization and representation strategy
-> target-independent runtime ABI
-> WebAssembly backend
-> WebAssembly host adapter
-> sol build
-> interpreter/WebAssembly differential suite
```

The production roadmap must explicitly cover generic monomorphization or dictionaries, aggregate and text layout, symbols and linkage, Wasm imports/exports, runtime memory management, panic and cleanup policy, source maps, artifact metadata, reproducibility, release CI, fuzzing, security review, and performance gates.

9 Roadmap Restructure

Recommended splits:

- Item 40: application metadata; dependencies and locks; visibility and re-exports; sandboxed builds.
- Item 48: interpreter host profile; target-independent runtime ABI; backend adapters.
- Item 49: entrypoint semantics; interpreter `sol run`; backend `sol build`.
- Item 50: normalized logical obligations, refinement-aware matching, and cost/resource checks beyond E5's runtime core.
- Item 43: immediate integration acceptance criteria rather than one permanent mega-task.

Recommended new explicit tasks:

- Versioned executable-core conformance suite.
- Shared compilation/session API.
- Compiler resource budgets.
- Host capability registry.
- Representative executable application.
- Monomorphization and data-layout strategy.
- Interpreter/backend differential testing.
- Fuzzing, allocation-failure testing, clean-build CI, and release hardening.

10 Deferred Work

For the interpreter-based application endpoint, defer closures and richer traits, broad numeric and collection support, full package dependencies, general effect polymorphism, unsafe, C FFI, MIR/backend work, SMT, public IR, language-server functionality, semantic patches, concurrency, reflection, and general resumptive handlers.

Only a minimal package/application profile is required before `sol run`.

11 Conclusion

The frontend/interpreter core is substantially complete and internally coherent. The project should pause language breadth.

The shortest route to demonstrable end-to-end value is:

1. Harden and centralize compilation.
2. Define entrypoint and host semantics.
3. Add `sol run` over the existing interpreter.
4. Make contracts executable.
5. Prove the workflow with one representative application.

Only after these semantics are stable should they be embedded into MIR and a WebAssembly or native backend.